Plain-English Guide to the VinePPO Testbed: Environments, Transformer Critic, and Fitted Backward Induction (FBI)

Project Overview & Visual Walkthrough

November 9, 2026

1 What This Project Is Doing (Big Picture)

When training a Large Language Model (LLM) to solve multi-step math or reasoning problems using Reinforcement Learning (RL), we usually only get a reward at the very end of the solution: $R = 1$ if the final answer is right, and $R = 0$ if it is wrong.

To train the model effectively with PPO (Proximal Policy Optimization), we need a **Critic** (value function $V_\phi(s_t)$) that can look at an incomplete reasoning chain at step t and estimate: *"What is the probability that the model will eventually reach a correct final answer from this point?"* Once we know the value of every step, we can tell whether a specific reasoning step a_t improved or hurt our chances of success via the step-level advantage:

$$\hat{A}(s_t, a_t) = r_t + \gamma V(s_{t+1}) - V(s_t).$$

Rather than running expensive LLM decoding experiments to study how best to train this critic, this project builds a fast, exact, pure-JAX synthetic testbed where we **know the exact ground-truth value $V^*(s)$ of every state** via dynamic programming, allowing us to directly measure how fast and accurately different RL value-learning algorithms converge.

2 The Environments (MRPs / MDPs)

The codebase models multi-step reasoning using two synthetic Markov environments where observations are sequences of tokens fed into a Transformer, and rewards are sparse and binary ($R \in \{0, 1\}$ at the end of the episode):

1. **Binary Tree MRP (`envs/tree_mrp.py`):**
   - **Structure:** A complete binary tree of depth D (e.g., $D = 4$ or $D = 7$) with $2^{D+1} - 1$ states. Starting at the root s_0, at each step the trajectory branches Left (`0`) or Right (`1`).
   - **Rewards & Dynamics:** Only a small cluster of leaf nodes at depth D are "correct solutions" yielding $R = 1$; all other leaves yield $R = 0$. At any internal node where one child subtree contains a correct leaf and the other is a dead end, the base policy picks the correct branch with probability p (e.g., $p = 0.8$) and blunders into the dead-end subtree with probability $1 - p$.
   - **Observation:** The sequence of binary choices taken so far, padded to length D (e.g., `[1, 0, 1, PAD, PAD]`).
2. **Procedural Reasoning DAG (`envs/reasoning_dag_env.py`):**

- **Why a DAG instead of a Tree?** Real reasoning is not a strict tree: different valid chains of thought can reconverge to the same intermediate milestone, and if a model makes a mistake (derails), it can verify/self-correct and get back on track.
- **Hidden Markov State:** Each problem requires completing M sequential **milestones** within T maximum steps ($T > M$). The true underlying state is tracked by the tuple (`prompt_idx`, step t, milestone $m \in \{0..M\}$, `is_derailed` $d \in \{0, 1\}$).
- **Transitions:** At each step, the base policy chooses from $|A|$ discrete action tokens. When on-track ($d = 0$), picking a valid advance token increments the milestone ($m \to m + 1$, probability $p_{\text{advance}} = 0.65$), picking a neutral token keeps ($m, d = 0$), and picking a blunder token derails the trajectory ($d \to 1$, probability $p_{\text{blunder}} = 0.15$). Once derailed ($d = 1$), normal actions fail to advance m; the trajectory can only return to $d = 0$ by emitting a special **self-correction/verification action** (probability $p_{\text{recover}} = 0.35$).
- **What the Critic Sees:** Crucially, the network **never** sees the privileged (m, d) variables. It only sees the raw sequence of emitted token IDs: `[BOS, PROMPT_ID, SEP,` a_0`,` a_1`, ...,` a_{t-1}`, PAD, ...]`. It must infer which milestone has been reached and whether the chain of thought is currently derailed purely from the token history.

In both environments, an exact backward-induction solver (`solve_backward_induction()`) precomputes the true value $V^*(s)$ and true action advantages $A^*(s, a)$ for every state so we can evaluate the critic's Mean Squared Value Error (MSVE) and Advantage Cosine Similarity at every training step.

3 The Network (`transformer.py`)

The critic $V_\phi(s_t)$ is a modern, miniature LLaMA-style causal Transformer (`TransformerCritic`):

1. **Token Embedding:** Maps the discrete observation sequence (and an optional prepended `BOS` token) into d_{model}-dimensional vectors (default $d_{\text{model}} = 64$).
2. **Causal Transformer Blocks ($N = 2$ layers, $H = 4$ heads):** Each layer applies Pre-RMSNorm, Multi-Head Causal Self-Attention with Rotary Position Embeddings (RoPE), and a SwiGLU feed-forward network.
3. **Last-Valid-Token Pooling:** Extracts the hidden state at the last non-padding token position (which summarizes the entire reasoning prefix up to step t).
4. **MLP & Value Head:** Passes that pooled vector through a 2-layer SiLU MLP with RMSNorm and a zero-initialized linear projection head to output a scalar value prediction $V_\phi(s_t) \in \mathbb{R}$.

4 The Value-Learning Algorithms

All algorithms collect a batch of $B = 64$ primary "base trajectories" $\tau = (s_0, a_0, s_1, \ldots, s_T)$ per update, but differ in how they construct the regression target $y(s_t)$ for the critic $V_\phi(s_t)$:

4.1 1. Standard Monte Carlo (`mc_value.py` / `mc`)

- **How it works:** Generates only the single base trajectory ($K = 1$). Every intermediate state s_t along that trajectory is trained to predict the final binary reward $R(s_T) \in \{0, 1\}$ of that single rollout.
- **Trade-off:** Cheapest rollout cost ($\mathcal{O}(T)$ transitions per trajectory), but highest variance: the same intermediate state s_t receives target 1 on one trajectory and 0 on another.

4.2 2. Vine-Target & Fitted Vine Interpolation (`vine_target_value.py` / `vine_k4`)

- **How it works:** Inspired by VinePPO, at *every* visited state s_t along the base trajectory, the environment state is cloned K times (e.g., $K = 4$) and rolled out **all the way to the end of the episode** ($t \to T$). The average terminal reward $\hat{V}_{\text{MC}}(s_t) = \frac{1}{K}\sum_{k=1}^{K} R(\tau_{t\to T}^{(k)})$ is used as a low-variance target to train $V_\phi(s_t)$.
- **Variant 2 (Precision-Weighted Shrinkage in `vine_target_value.py`):** Because $\hat{V}_{\text{MC}}(s_t)$ is an average of K Bernoulli trials, we know its exact sampling variance $\hat{\sigma}^2_{\text{MC}}(s_t) = \frac{\hat{V}_{\text{MC}}(1-\hat{V}_{\text{MC}})}{K-1}$. Before updating the critic on each batch, the code audits how much excess error σ^2_{critic} the network V_ϕ has beyond $\hat{\sigma}^2_{\text{MC}}$, and blends $\hat{V}_{\text{MC}}$ and V_ϕ using inverse-variance weights.
- **Trade-off:** Completely unbiased targets (zero bootstrapping drift), but requires $\mathcal{O}(K \cdot T^2)$ environment steps per trajectory because early states must be rolled out all the way to horizon T.

4.3 3. Fitted Backward Induction — FBI(λ) (`fbi_value.py` & `run_benchmark.py`)

Why FBI exists: Instead of rolling out K branches all the way to the end of the episode ($\mathcal{O}(K \cdot T^2)$ cost), FBI rolls out K branches for **only 1 step** ($s_t \to s_{t+1}^{(1..K)}$) at each state along the base chain ($\mathcal{O}(K \cdot T)$ cost), and uses the critic V_ϕ itself to evaluate those 1-step children.

Because FBI relies on its own network $V_\phi(s_{t+1}^{(k)})$ to create targets for s_t, the order in which rollouts and gradient updates happen is critical. Notice that the codebase actually contains **two implementations** of FBI:

A. The Canonical Sequential Backward-Induction Loop (`fbi_value.py` & `fbi_ppo.tex`)

In `fbi_value.py`, each outer training iteration consists of **two rollout steps** followed by D **sequential rounds (stages) of learning**:

1. **Rollout Phase (Done once per outer iteration before fitting):**
   - *Rollout Step 1 (Base Chain):* Roll out $B = 64$ primary trajectories from root s_0 to leaf s_D ($B \times D$ transitions).
   - *Rollout Step 2 (Parallel 1-Step Branches):* From every state s_t ($t \in \{0, \dots, D-1\}$) along the base chain, fork $K = 4$ parallel **1-step** transitions $s_{t+1}^{(1..K)}$ ($B \times D \times K$ transitions). Note that for the penultimate node s_{D-1}, a 1-step rollout lands directly on terminal leaves $s_D^{(1..K)}$ where exact rewards $R \in \{0, 1\}$ are observed!
2. **How Many Rounds of Learning? Does It Learn All-at-Once or in Reverse Order?**
   - It learns in **reverse backward-induction order**, across D **sequential fitting stages** per outer iteration, running `STAGE_STEPS = 4` gradient updates at each stage ($4 \times D$ gradient steps total per iteration).
   - **Crucial detail — Cumulative Replay Anchoring (Not isolated 1-level fitting!):** If you trained a neural network *purely* on level t alone at each backward step, gradient descent on level t would overwrite and destroy the weights that just learned levels $t+1 \dots D$ (catastrophic forgetting). Therefore, `fbi_value.py` uses a **growing cumulative mask** (`active_mask` / $\mathcal{D}_{\text{fit}}$):
     - **Stage 0 (Initial Base Case at the end of the DAG/Tree, $t = D$ and $t = D-1$):** We know the exact reward $y_D = R(s_D)$ at the leaf and the K-sample Monte Carlo average $y_{D-1} = \frac{1}{K}\sum_{k=1}^{K} R(s_D^{(k)})$ at the penultimate node. We put levels $\{D, D-1\}$ (plus all K branched leaf states $s_D^{(1..K)}$) into $\mathcal{D}_{\text{fit}}$ and run **4 gradient steps** (`stage_steps = 4`). Now V_ϕ is accurate at the end of the horizon.

– **Backward Loop ($t = D-2$ down to 0):** At level t, we use the **just-updated** network V_ϕ to evaluate the K parallel 1-step children $s_{t+1}^{(1..K)}$:

$$\hat{V}_{\text{branch}}(s_t) = \frac{1}{K} \sum_{k=1}^{K} \text{sg}\Big(V_\phi(s_{t+1}^{(k)})\Big).$$

If $\lambda = 0$, the target is simply $y_t = \hat{V}_{\text{branch}}(s_t)$. More generally, FBI($\lambda$) blends this 1-step branched expectation with downstream targets using a reverse GAE accumulator over branched residuals $\delta_t^{\text{branch}} = \hat{V}_{\text{branch}}(s_t) - V_\phi(s_t)$:

$$\hat{A}_t^{(\lambda)} = \delta_t^{\text{branch}} + (\gamma\lambda)\hat{A}_{t+1}^{(\lambda)}, \qquad y_t^{(\lambda)} = \text{sg}(V_\phi(s_t)) + \hat{A}_t^{(\lambda)}.$$

We then **append** $(s_t, y_t^{(\lambda)})$ to the cumulative buffer $\mathcal{D}_{\text{fit}}$ (which now holds $\{t, t+1, \ldots, D\}$) and run **4 gradient steps** on $\mathcal{D}_{\text{fit}}$ before moving backward to $t-1$.

B. The Batched Synchronous Variant (`run_benchmark.py`)

In `run_benchmark.py` (used for rapid multi-seed sweeps on the 24–32 step `ReasoningDAGEnv`), running $T = 32$ separate inner fitting stages inside every outer loop would require $32 \times 4 = 128$ sequential passes per rollout batch. Instead, `run_benchmark.py` implements a **batched all-at-once approximation**:

1. Collects the primary chain $(s_0 \ldots s_T)$ and forks $K = 4$ parallel 1-step branches at every node.
2. Evaluates $\hat{V}_{\text{branch}}(s_t) = \frac{1}{K}\sum_{k=1}^{K}\left(r_t^{(k)} + \gamma V_\phi(s_{t+1}^{(k)})\right)$ across all steps simultaneously using the current network V_ϕ.
3. Sweeps backward in reverse order ($t = T-1 \to 0$) in a single fast vector pass to compute the FBI(λ) GAE targets $y_t^{(\lambda)}$.
4. Trains the network **on all timesteps** $\{0 \ldots T-1\}$ **at once** for `NUM_EPOCHS = 4` shuffled minibatch epochs. Here, the backward propagation of value across the DAG happens across outer training updates (and via $\lambda > 0$ within each update).

5 Visualizing Fitted Backward Induction (FBI) on a Reasoning DAG

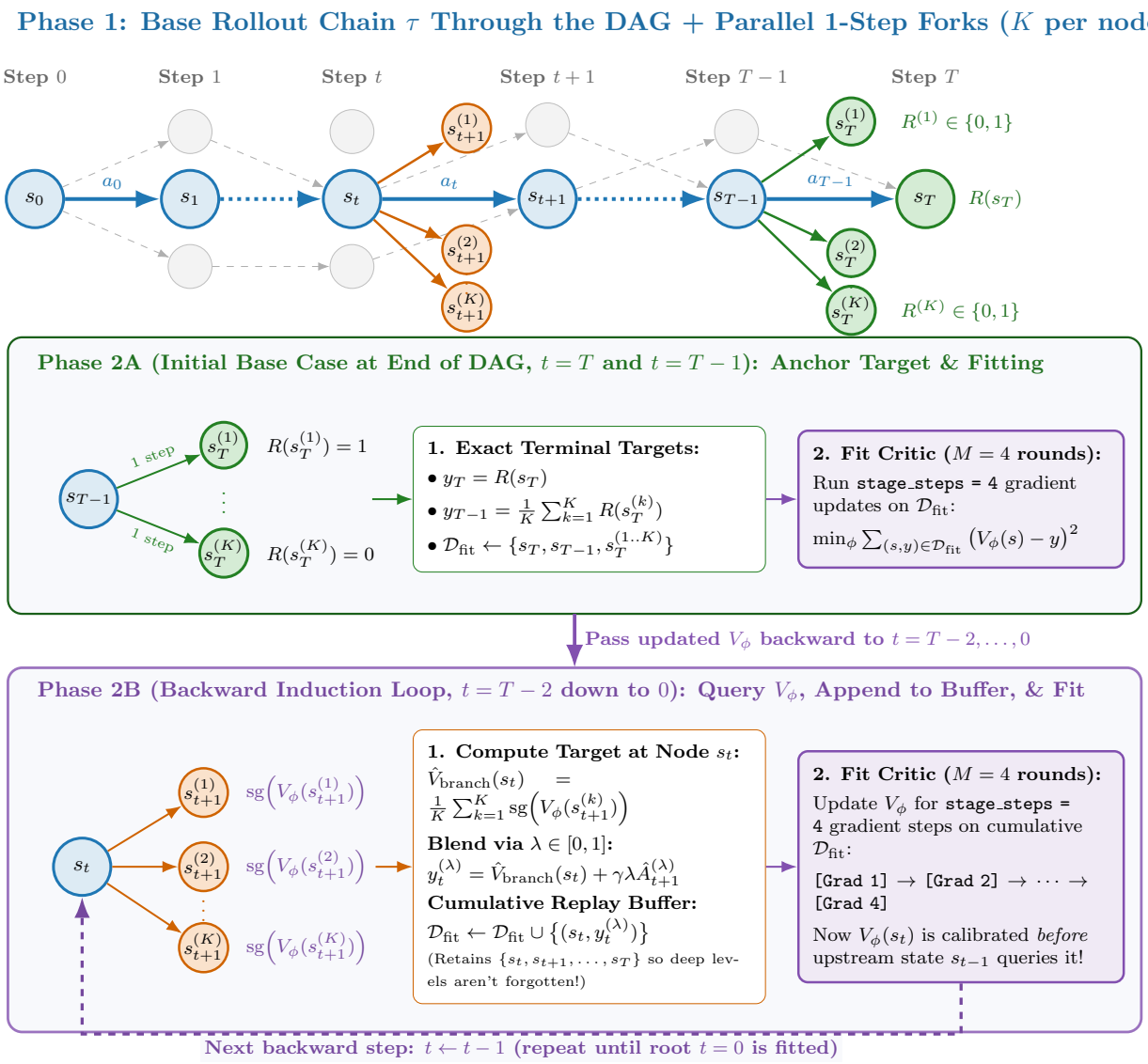


Figure 1: **How 1-Step Fitted Backward Induction (FBI(λ)) Solves the DAG Value Problem. Top (Phase 1):** A primary trajectory $\tau = (s_0 \ldots s_T)$ is rolled out as a chain, and at every visited state s_t, K parallel 1-step rollouts $s_{t+1}^{(1..K)}$ are sampled. **Middle (Phase 2A):** Learning begins at the end of the DAG ($t = T$ and $t = T-1$), where the 1-step branches land on terminal leaves with exact rewards $R^{(1..K)}$, and the critic V_ϕ is fitted for $M = 4$ gradient rounds. **Bottom (Phase 2B):** Sweeping backward from $t = T-2$ down to 0, the intermediary node s_t queries the *just-updated* critic V_ϕ on its K parallel 1-step children, appends $(s_t, y_t^{(\lambda)})$ to the cumulative replay buffer $\mathcal{D}_{\text{fit}}$ (retaining $t' \geq t$), and runs $M = 4$ fitting rounds before stepping backward to $t-1$.